

Enhancing Secure Data Transmission through Deep Learning-based Image Steganography and Super-Resolution Generative Adversarial Networks

Sreyas Ramesh, B N Sukanth, Sathyavarapu Sri Jaswanth, Rimjhim Padam Singh*

Department of Computer Science and Engineering

Amrita School of Computing, Bengaluru

Amrita Vishwa Vidyapeetham, India

bl.en.u4aie21120@bl.students.amrita.edu, bl.en.u4aie21014@bl.students.amrita.edu,

bl.en.u4aie21116@bl.students.amrita.edu, ps_rimjhim@blr.amrita.edu*

Abstract—In the era of multimedia technology digital images are essential and keeping them safe from unauthorised access is crucial. To address this the research explores the intersection of image steganography and deep learning for enhanced secure data transmission and proposes a unique approach that leverages deep learning for feature extraction and steganography using the Least Significant Bit (LSB) method with Stego sprite image formation. Custom deep learning architectures are investigated, including simple convolutional autoencoders, VGG-inspired deeper convolutional networks, and ResNet-inspired skip connection based models, to draw a comparison on the performance of the models to extract semantic feature and reconstruct images from the semantic features. Furthermore, a Super-Resolution Generative Adversarial Network (SRGAN) is employed to improve the quality of the reconstructed images. The best performing model was the Skip-connection based model inspired by ResNet which gave a SSIM value of 1.00 and PSNR value of 33.04. Overall, This research combines deep learning with steganography to create a secure and hidden communication method.

Index Terms—Deep Learning, Steganography, Least Significant Bit, Sprite image, Super-Resolution Generative Adversarial Network, Data Security

I. INTRODUCTION

In today's digital landscape, safeguarding sensitive information from unauthorized access is a critical challenge. The concealment and secure transmission of data pose significant hurdles, necessitating inventive solutions. This study delves into merging image steganography with profound deep learning techniques to effectively tackle these imperatives.

The rise of digital communication has heightened concerns about data security and privacy breaches. While traditional encryption methods are effective, they still pose vulnerabilities that can be exploited. This underscores the importance of covert communication techniques that hide sensitive information, reducing the risk of interception and unauthorized access.

To address these challenges, we harness the capabilities of deep learning for feature extraction and image reconstruction. We explore custom architectures like custom Simple Convolutional Autoencoders inspired by CAE, Deeper Convolutional

network inspired by VGG16, and Skip Connection based model inspired by ResNet50, each offering unique strengths in extracting semantic features from images and reconstructing hidden data. Through this approach, we aim to enhance the efficiency and effectiveness of steganographic techniques in secure data transmission.

At the core of our methodology is the fusion of steganography with deep learning. We utilize the Least Significant Bit (LSB) embedding method, a classic steganographic technique, to seamlessly hide data within host images. By embedding serialized semantic features within pixel values of seemingly non secret images using LSB, we ensure invisibility to the human eye while enabling discreet transmission.

Moreover, we emphasize the importance of improving the quality of reconstructed images for better interpretation by introducing the Super-Resolution Generative Adversarial Network (SRGAN), we leverage adversarial training to enhance the fidelity and resolution of reconstructed images, improving visual clarity and detail. In essence, this study proposes an innovative integration of image steganography and deep learning to address the critical need for secure data transmission. By exploring advanced feature extraction techniques, traditional steganographic methods, and profound image enhancement approaches, we aim to establish robust and covert communication paradigms in the digital realm.

In the subsequent sections of this paper, in section 2 we delve into a thorough review of related works in the field of image steganography and GAN's, exploring methodologies utilized in similar studies. Following related works is the "Dataset Description" section 3, gives a gist about the dataset we acquired and then the "Proposed Methodology" section 4, outlines our approach, employing and exploring the combination of custom Deep learning models and steganography along with SRGAN for image enhancement. Subsequently, the "Results" section 5 presents our findings and the outcomes of the methodology. Finally, the "Conclusion" section 6 summarizes our key findings, discusses the implications of our results, and throws light on future research directions.

II. LITERATURE SURVEY

Zhenjie Bao and team [1] discuss the recent advancements in image encryption techniques have witnessed the integration of deep learning methodologies, particularly Cycle-GAN, to address challenges such as the weak avalanche effect observed in traditional encryption schemes. In response to this, they have proposed novel hybrid encryption algorithms that combine traditional diffusion techniques with deep learning neural networks to enhance encryption strength. These algorithms have shown promise in effectively scrambling and diffusing pixel values in various types of images, including satellite imagery. On the other hand Liu and Xue [2] introduce a model that not only guarantees security of content but also the safety of visual images through a combination of compressed sensing (CS) and Cycle-GAN. By leveraging discrete wavelet transform and Henon map, plaintext images are transformed into a sparse domain before being encrypted using Cycle-GAN. The reconstruction process with GAN ensures the original form of the image is retained, thereby maintaining encryption quality and robustness against attacks. Additionally, Panwar et al. [3] discuss the application of generative adversarial networks (GANs), particularly Cycle-GAN, in image encryption schemes. Their proposed encryption scheme, Encipher GAN, employs deep convolutional neural networks to transform plaintext images into cipher images and vice versa. By incorporating loss functions like SSIM, the scheme achieves high-quality image recovery while ensuring security against cryptanalysis, particularly in sensitive domains such as medical imaging. Furthermore, Kuyoro et al. [4] propose a model utilizing Cycle-GAN and DCGAN for reverse image steganography, focusing on hiding secret images within cover images. By leveraging Cycle-GAN's ability to train without requiring translated images and preventing overfitting, along with DCGAN's U-net-based generator architecture, the model successfully encodes secret images into cover images. Evaluation metrics such as PSNR, SSIM, and BPP demonstrate the effectiveness of Cycle-GAN in comparison to DCGAN. Similarly, Ramaneti et al. propose a methodology for implementing image steganography using GANs, specifically DCGAN [5]. The model involves training a DCGAN to generate stego-images based on noise, which are then transmitted to a receiver. A CNN based extractor is trained to remove the noise from the stego-images thereby leading to the extraction of the original image.

Ding et al. [6] introduce DeepKeyGen, a streaming cipher generator, which combines stream ciphers with GAN-based key generation to safeguard medical images. Similarly, Singh et al. [7] propose an encryption model that employs GAN-generated encryption keys alongside substitution, permutation, and diffusion algorithms. These approaches demonstrate resilience against decryption attempts without both the key and model. Chahar et al. [8] provide a comprehensive review of deep learning approaches in image steganography, emphasizing the increasing adoption of GANs to enhance image security and concealment. Li et al. [9]

propose a scheme that integrates chaos encryption with GANs to address challenges in grayscale image steganography, ensuring high embedding capacity while preserving image quality. Additionally, Qin et al. [11] explore coverless steganography using GAN-based encoding of messages into cover images without altering their appearance, achieving high hiding capacity and efficiency. Sharma et al. [12] propose an image encryption approach that combines steganography with GANs, utilizing the trained GAN model as a key for encryption and decryption. This symmetric encryption method offers potential for securing complex images with a larger number of pixels. Similarly, Yu et al. [13] present a methodology to enhance information recovery accuracy and steganography capacity using GANs. By optimizing training modes and loss functions and incorporating attention mechanisms and soft margin discriminators, the proposed methodology achieves superior results in image recovery and steganography capacity.

Yendluri Lohith Jayasurya et al. [14] address the vulnerabilities of traditional steganographic techniques and propose a CNN-based approach to hide secret images within cover images. By training the CNN to effectively conceal secret images, they demonstrate the potential of neural networks in revolutionizing steganography. The authors suggest future research directions including leveraging pretrained models for steganalysis and integrating adversarial networks to reduce errors. Similarly, Yuan et al. [15] propose a steganographic scheme that resists detection by CNN-based steganalyzers. Their method utilizes GAN, pixel-wise fusion, and adversarial attack modules to ensure security. By employing Wasserstein GAN model and conducting adversarial training, better performance and results were achieved thus, enhancing data security.

P. Mohan et al. [16] propose a spatial domain steganography method utilizing Least Significant Bit (LSB) embedding and Block Truncation Coding (BTC). By embedding messages into the least significant bit of carrier images and compressing the negative of the input using BTC, the authors achieve visually undetectable changes in the cover image. Similarly, M. Kataria and team [17] explore advanced steganography techniques combined with encryption algorithms for secure image transmission. By integrating encryption algorithms such as ESA, DSE, RSA, and ChaCha20 with LSB and Spread Spectrum methods, the authors assess the smoothness, sharpness, and security of the stego-images produced.

Martin et al. [18] propose an enhanced image steganography method utilizing GANs through three experiments aimed at evaluating GAN architecture performance and the effectiveness of combining GANs with genetic algorithms to deceive steganalysis models. Their future directions include exploring other steganography algorithms and lighter GAN architectures for diverse cover mediums. Similarly, Hafiz Abdul Rehman et al. [19] introduce a novel approach to coverless image

steganography by utilizing GANs to hide secret information within anime characters. Their methodology involves character generation, utilizing LSTM and SRGAN models for text to face attribute labelling and high-resolution image generation. The proposed scheme significantly boosts hiding capacity and security, thereby achieving high scores in various evaluation metrics. On the other hand, Ye Yao and team [20] discuss the emergence of deep learning-based steganography methods, combining Discrete Wavelet Transform (DWT) and GANs to enhance embedding capacity and security. Their proposed method introduces a multi-scale attention convolution module and a fusion module for improved feature capture and embedding location accuracy. The novel design gives us a contrast between the cover and steg images patch-wise, enhancing security. Similarly, Wang and team [21] propose an approach for image steganography that is based on cost learning GAN. To simulate the information embedding process they introduce a discriminator, generator, and a double-tanh function. Their method incorporates an attention mechanism for detail enhancement, a dual-stream input for feature preservation, and skip interconnections for information flow adjustment.

III. DATASET DESCRIPTION

A. CIFAR-100 image dataset

CIFAR-100 dataset is primarily designed for object recognition task, it consists of 60,000 color images of size 32x32. The training and the testing division is 50,000 and 10,000 respectively. CIFAR-100 has 100 object classes as the name suggests, each class is split where 500 images are for training and 100 images are for testing testing images. This dataset was employed to train the deep learning models for extracting and reconstructing image semantic features.

B. Image Super Resolution dataset

A dataset to train a model to convert low quality images to high quality images. The dataset is divided into 2 main folders, 'train' and 'val'. Inside these 2 folders, there are sub directories called high_res & low_res which correspond to high-resolution images and low-resolution images respectively, both are of sizes 256x256. The 'train' folder consists a total of 1370 images, 685 images each for high resolution and low resolution. The 'val' folder consists a total of 340 images, 170 images each for high resolution and low resolution. This dataset was employed to train the SRGAN for generating higher quality images.

IV. PROPOSED METHODOLOGY

The methodology consists of 4 major steps which includes Extracting Semantic Features, using steganography to hide the image, reconstructing the image back after retrieving the semantic features and computing the evaluation metrics. The model architecture can be referred to in figure 1 & figure 2 and the explanation of each steps follows.

A. Extracting Semantic features

1) Encoder Model Architecture :

a) *Convolutional Layers*: The encoder layer consists of series of convolutional layers that slide over the input image. Each layer extracts the spatial patterns or features present in the input. The model is trained using CIFAR-100 which is a dataset of images. The encoder model learns to detect edges, visual patterns and textures of the images given for training.

b) *Max-Pooling layers*: Max pooling layers are applied after every convolutional layers which helps in reducing the spatial dimensions of the feature maps, in these layers the most important features are retained within a small window. The purpose of the max pooling layer is to reduce the

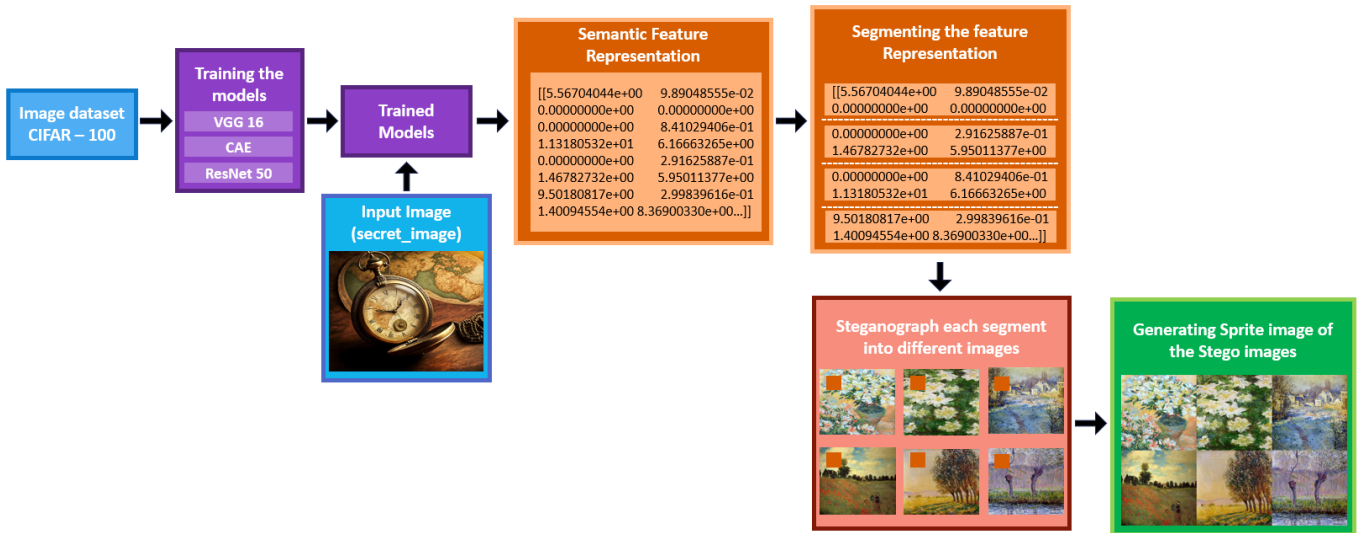


Fig. 1. Model Flow Architecture for Semantic Feature Extraction

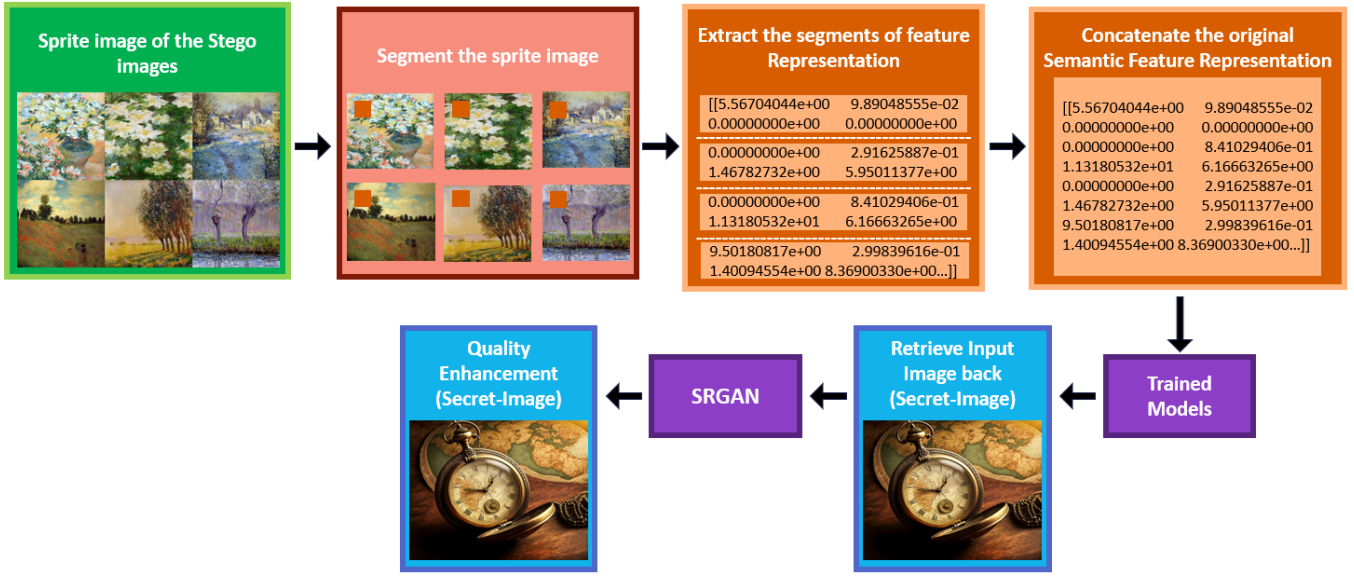


Fig. 2. Model Flow Architecture for extracting Embedded segments and image Reconstruction and Enhancement

computational load and make the network more robust to different variations in the input.

c) *Feature Extraction:* The combination of convolutional and max-pooling layers are stacked over other which form a deep architecture. This architecture helps in capturing properties like edges and texture which are handled by lower level layers, while the more complex features which include the object shapes, abstract features and structures are handled by higher layers.

2) Pre-processing the input Image:

a) *Resizing the image:* The input images are resized to a fixed size so that it becomes compatible with the encoder model the size used in the model is (32, 32). Resizing is an important pre-processing step as many neural networks have fixed input sizes. During the resizing process it is important to maintain the aspect ratio to prevent distortions or noisy image.

b) *Normalization:* Many images go through normalization pre-processing technique which is used to scale pixel values to a specific range, typically [0,1] or [-1,1]. Normalization can help improve the convergence speed of many deep learning models. The images are represented in the form of 8-bits ranging from (0 to 255). This makes sure that the pixel values of the images fall within the range [0,1].

c) *Formatting as batch:* While using deep learning models, the inputs are formatted into batches, which helps the models to process single samples. This batch dimension allows for efficient optimization and efficient parallel processing. While formatting the input into batches there is an inclusion of an addition axis at the beginning.

3) Forward pass through encoder:

a) *Convolutional operations and Max Pooling:* The processed image batches are passes through the encoder model which consists of multiple convolutional and max-pooling layers. The convolutional layers consists of a set of learnable filter which is applied in every layer. The learnable parameters slides over the input image thereby evaluating the dot product at different parts of the input data. The convolutional layer as explained above extract features from the input images. During training the model adjusts the values of the filters/kernels of the convolutional layers to minimize the distance between the predicted output and the ground truth labels. The max-pooling layers are used to reduce the size of the features and retaining the most important information. This helps in making the representations more compact and reduces computational complexity.

b) *Semantic features:* The encoder provides output that is a compressed representation of the input image which is known as the semantic features. The representation gives a summary of the essential information about the image in compact form. The features represent the most important information of the image which includes the characteristics of the image, while discarding repeating and unimportant information. The semantic features are further sent in as input for subsequent layer for steganography as well as image reconstruction. The encoder model enables the network to perform more efficiently, when the input images are in compact representation in the form of semantic features.

B. Image Steganography

Steganography is the technique of hiding data within another non-secret multimedia forms such as image, audio or video. The concealed information is very well hidden and is not visible to the observer.

1) *Semantic Features Serialization*: The semantic feature representation of the secret image obtained by passing through the trained deep learning models are of the form of 4-dimensional array on the sender's side. The 4-dimensional array form of the semantic features is converted to the form string with JSON. JSON typically works well with Python lists, so the 4-dimensional NumPy array was converted into a Python list. This step ensures that the data can be easily serialized into JSON format. Then a dumps function was used to serialize the list into a JSON-formatted string.

2) *Embedding Message into image*: Next, the JSON string representing semantic features undergoes fragmentation or splitting into discrete parts. Each fragment is concealed within separate images utilizing the Least Significant Bit (LSB) steganography technique. The segments of the string are converted binary and is hidden with the least significant bits. This process will create a set of images with each image having different segments of the JSON string hidden within.

3) *Creation of Sprite image*: The amalgamation of images, each containing fragments of the JSON string, lead to the formation of a composite sprite image that is a collaged image. First the grid size is determined based on the number of embedded images and then the images are collaged together to form a single sprite image. This sprite image serves as the comprehensive carrier of the hidden information and facilitates efficient storage and transmission.

4) *Extraction of Embedded Messages*: On receiver's side During the extraction phase, the sprite image is segmented into individual images, each containing a portion of the embedded message. Each segment of the embedded message is extracted from each individual image by applying the reverse process of extracting the hidden data within the least significant bits. These messages are then concatenated to reconstruct the original JSON string representing semantic features.

5) *Recovery of Semantic Features*: The reconstructed JSON string is deserialized back into a list of semantic features. Starting with deserializing the reconstructed JSON string into a list followed by converting the list back into a NumPy array representing the semantic features, completing the recovery process. Now the recovered feature representation shall be used to reconstruct the secret image back on receiver side via trained models.

C. Image Reconstruction

1) Decoder model architecture:

a) *Sequence of layers*: The decoder structure follows the structure and sequence similar to that of the encoder, these layers include convolutional layers, upsampling, activations and normalization layers.

b) *Upsampling Layers*: The upsampling layers help increasing the feature maps dimension allowing the decoder to reconstruct the image at higher resolution. Upsampling layers help to restore spatial details which were lost during the process of downsampling thereby helping the reconstruction process.

c) *Convolutional Layers*: The convolutional layers in the decoder apply filter which learn the upsampled features, these filters transform the feature maps thereby reconstructing the image by extracting features from the encoder.

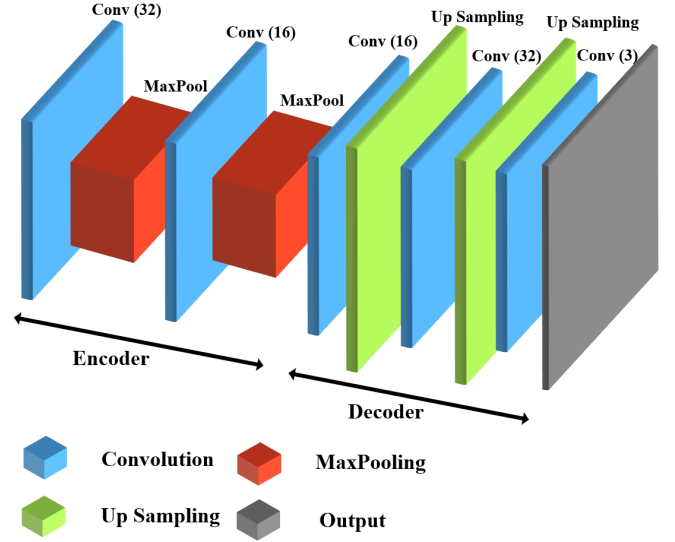


Fig. 3. Custom Simple Convolutional Auto Encoder Model Architecture

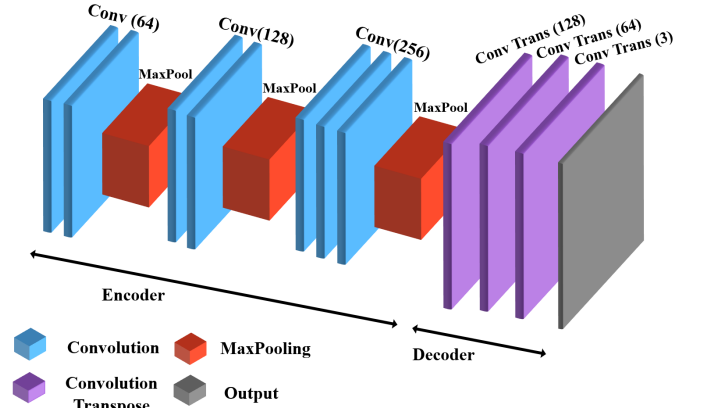


Fig. 4. Deeper Convolutional Network Model Architecture

2) *Reconstruction Process*: The decoder is a combination of both upsampling and convolutional layers which are repeated across multiple layers in the decoder. As the decoder goes through each layer the features are combined from the previous layers thereby refining the features extracted from the encoder. The final output of the decoder is reconstructed image that resembles the original image. The model Architecture for Simple convolutional autoencoder can be referred to in Fig. 3.

Here, three traditional deep learning models have been explored and modified for our work which are custom Simple Convolutional auto encoder inspired by CAE, Deeper Convolutional network inspired by VGG16, and Skip Connection based model inspired by ResNet50. In simple convolutional

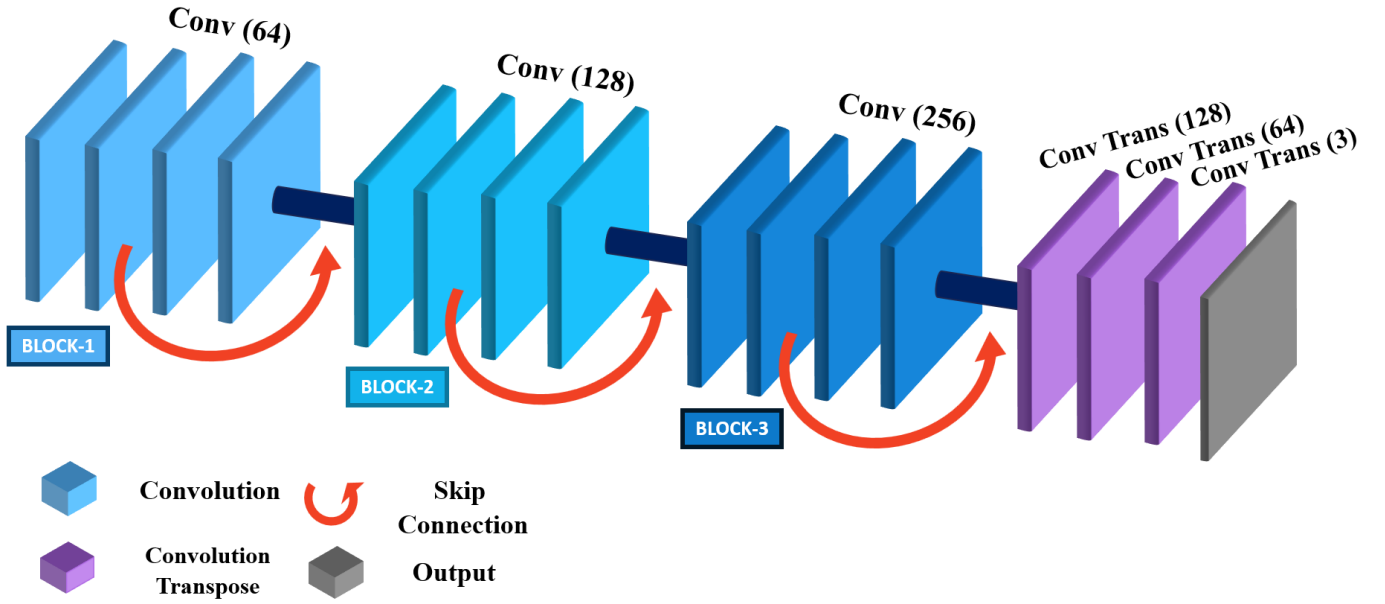


Fig. 5. Skip Connection based Network Model Architecture

auto encoder there are 4 layers in encoding phase and 5 layers in decoding phase have been used. The model Architecture for Deeper Convolutional network can be referred to in Fig. 4.

In Deeper Convolutional network the first 3 blocks(10 layers) of traditional VGG16 model have been used as encoder phase and in decoder phase 3 convolutional transpose layers have been used. In Skip Connection based model unlike the traditional ResNet50, three stages with 1 block in stage(7 layers for each block followed by skip connection) total 21 layers have been used in encoder phase and 3 convolutional transpose layers have been used in decoder phase. The model Architecture for Skip Connection based model can be referred to in Fig. 5.

D. Evaluation Metrics

1) *Mean Squared Error (MSE)*: MSE helps in measuring the average squared difference between the pixels in the original input and reconstructed images. The MSE is computed from the flattened 1D array of both input and reconstructed images. Then the squared difference between the pixels are calculated and the mean is taken between them.

2) *Mean Absolute Error (MAE)*: Similar to MSE the original and reconstructed images are flattened to 1D arrays, the absolute differences between pixel values are found. MAE measures the average absolute differences between corresponding pixels between the original and reconstructed image.

3) *Peak Signal to Noise ratio (PSNR)*: PSNR is a measure which shows the reconstruction quality in terms of signal to noise ratio. The computation of PSNR is based on maximum pixel values and the mean squared error between the original input and reconstructed images. PSNR is expressed in form of decibels.

4) *Structural Similarity index (SSIM)*: SSIM gives a visual measure of image quality by comparing with luminance, contrast and structural similarities between original input and reconstructed images. SSIM values typically lie from the range -1 to 1.

E. Super-Resolution Generative Adversarial Networks (SRGAN)

Low-resolution and corresponding high-resolution image pairs are loaded from the specified directories. The loaded images are resized and transformed using PyTorch's transforms.

1) *Generator Network Forward Pass*: A low-resolution image is passed through the generator network, which consists of various convolutional, residual, and upsampling layers. The generator network generates a super-resolved, high-resolution image from the low-resolution input.

2) *Discriminator Network Forward Pass*: Both the generated high-resolution image from the generator and the ground truth high-resolution image are passed through the discriminator network, which consists of convolutional and fully connected layers. The discriminator network outputs a probability score indicating whether the input image is real or fake.

3) Loss Computation:

a) *Adversarial Loss*: The adversarial loss is computed using the binary cross-entropy loss between the discriminator's output for the generated image and the desired label (real or fake).

b) *Perceptual loss*: The perceptual loss is computed by passing the generated and ground truth high-resolution images through a pretrained VGG19 network and calculating the mean squared error between their feature representations.

c) *Mean Squared Error (MSE) Loss*: The MSE loss is computed between the generated and ground truth high-resolution images to ensure pixel-wise similarity

4) Backpropagation and Optimization:

a) *Discriminator Update*: The discriminator network is updated by maximizing the log likelihood of correctly classifying real and fake images (minimizing the adversarial loss).

b) *Generator Update*: The generator network is updated by minimizing the combined loss, which includes the adversarial loss and the perceptual loss.

The model's performance is evaluated on a validation dataset by computing the perceptual loss between the generated and ground truth high-resolution images. The generator and discriminator models are saved at regular intervals during training. The plot_examples function is used to visualize the low-resolution input, the generated high-resolution output, and the ground truth high-resolution image. This process is repeated for multiple epochs, with the generator and discriminator networks being trained adversarially to improve the quality of the generated high-resolution images.

V. RESULTS

As formulated in the methodology, steganography technique was integrated with three deep learning models that used the encoder and decoder architecture. The PSNR and SSIM values were evaluated for the original image and embedded image using LSB technique, which gave us the result 24.48 and 0.95 respectively, which is also supported with the fig(8). The results shows the embedding model has been implemented efficiently and in Fig. 6 the Training and Validation loss plots can be observed for the three models implemented. In Fig. 7 the generated sprite image can be observed. Each of the image collaged in the sprite image carries a piece/part of the semantic feature of original image steganographed.

Table 1 gives us the comparison between the 3 deep learning models on test data for image reconstruction using the evaluation metrics PSNR, SSIM, MAE, and MSE.

TABLE I
COMPARISON OF THE EVALUATION METRICS FOR IMAGE
RECONSTRUCTION ON TEST DATA USING 3 IMPLEMENTED MODELS

Evaluation Metrics	Simple CAE	Deeper CNN	Skip Connection
SSIM	0.97	0.98	1.00
PSNR	25.25	26.23	33.04
MAE	0.0382	0.0345	0.0160
MSE	0.0030	0.0024	0.0005

Table 2 gives us an overview of the comparison of three deep learning models for resized and reconstructed image using the same set of evaluation metrics. Table 3 gives us an overview of the comparison between the same deep learning models for original image and resized reconstructed image using the same evaluation metrics.

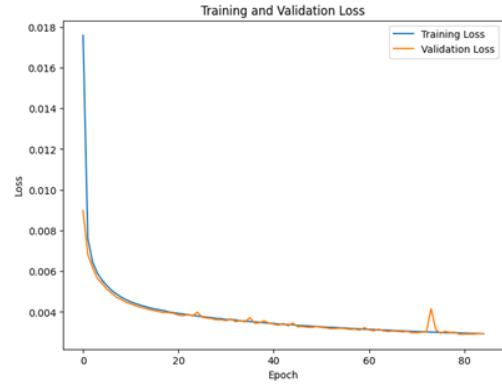


Fig. 6. Simple Convolutional Auto Encoder Training and Validation Loss Plot

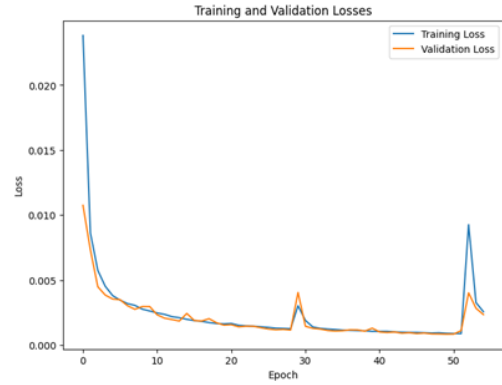


Fig. 7. Deeper Convolutional Network Training and Validation Loss Plot

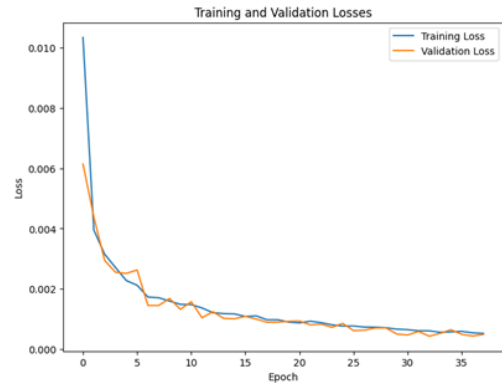


Fig. 8. Skip Connection Based model Training and Validation Loss Plot

TABLE II
COMPARISON OF THE EVALUATION METRICS FOR BETWEEN RESIZED AND RECONSTRUCTED IMAGE VIA THE 3 IMPLEMENTED MODELS

Evaluation Metrics	Image 1			Image 2		
	Simple CAE	Deeper CNN	Skip Connection	Simple CAE	Deeper CNN	Skip Connection
SSIM	0.64	0.70	0.87	0.73	0.78	0.91
PSNR	21.28	21.84	28.09	20.48	21.17	27.84
MAE	0.0548	0.0486	0.0266	0.0603	0.0559	0.0273
MSE	0.0074	0.0065	0.0016	0.0090	0.0076	0.0016

TABLE III
COMPARISON OF THE EVALUATION METRICS FOR ORIGINAL IMAGE AND RESIZED RECONSTRUCTED IMAGE VIA THE 3 IMPLEMENTED MODELS

Evaluation Metrics	Image 1			Image 2		
	Simple CAE	Deeper CNN	Skip Connection	Simple CAE	Deeper CNN	Skip Connection
SSIM	0.8063	0.8232	0.8295	0.7713	0.7890	0.8134
PSNR	20.39	20.61	21.75	19.18	19.50	20.91
MAE	142.7451	121.5309	92.9238	154.8171	134.6165	84.8704
MSE	68.5119	60.9709	51.9316	71.0782	69.8200	53.1718

TABLE IV
COMPARISON OF THE EVALUATION METRICS FOR ENHANCEMENT OF TEST IMAGES VIA SRGAN USING THE 3 IMPLEMENTED MODELS

Evaluation Metrics	Image 1			Image 2		
	Simple CAE	Deeper CNN	Skip Connection	Simple CAE	Deeper CNN	Skip Connection
SSIM	0.3734	0.3264	0.2943	0.5754	0.5851	0.5944
PSNR	17.5918	17.2877	17.3854	17.4192	17.7356	18.2128
MAE	0.1021	0.1066	0.1064	0.0898	0.0858	0.0788
MSE	0.0174	0.0187	0.0183	0.0181	0.0168	0.0151



Fig. 9. Generated Sprite Image

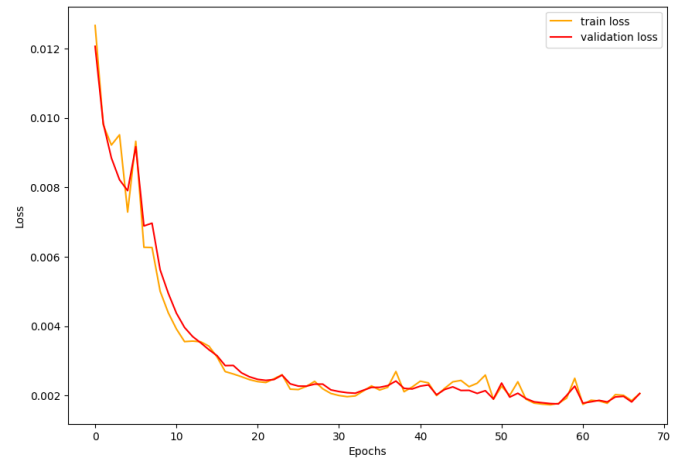


Fig. 10. SRGAN Training and Validation Loss Plot

Table 2 and 3 can be analysed with reference to Fig. 11, Fig. 12 and Fig. 13 which gives us a visualization of reconstruction process of three custom input images for three deep learning models. After a cumulative observation it can be inferred that the best performing model is Resnet 50 with SSIM value ranging from 0.87 to 0.91 for resized and reconstructed image and 0.813 to 0.829 for original image and resized reconstructed image. The PSNR values for the same include 27.84 to 28.09

for resized and reconstructed and 20.91 to 21.75 for original and resized reconstructed image. SRGAN was used to enhance the images further which improved SSIM values to 0.90 to 0.95. Fig. 14 gives us the visualisation of the performance of SRGAN on test data and the custom input visualisation of the performance of SRGAN can be observed in Fig. 15

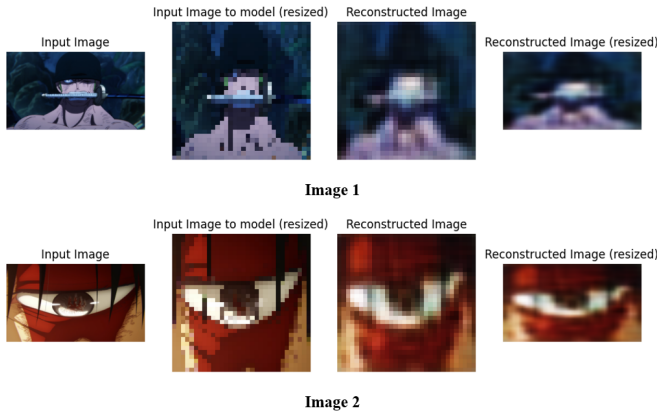


Fig. 11. Comparison of 2 custom test images between Original and Reconstructed images for CAE

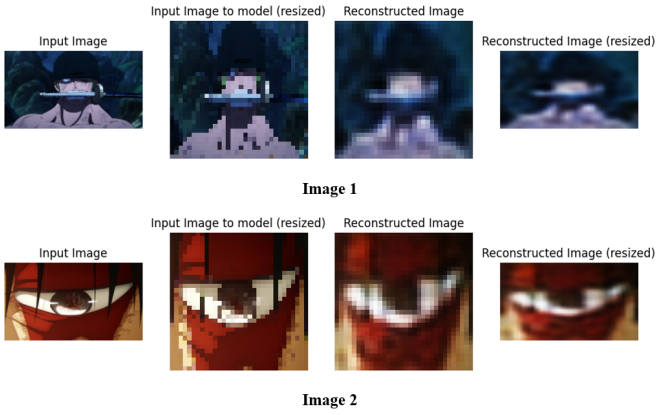


Fig. 12. Comparison of 2 custom test images between Original and Reconstructed images for VGG16

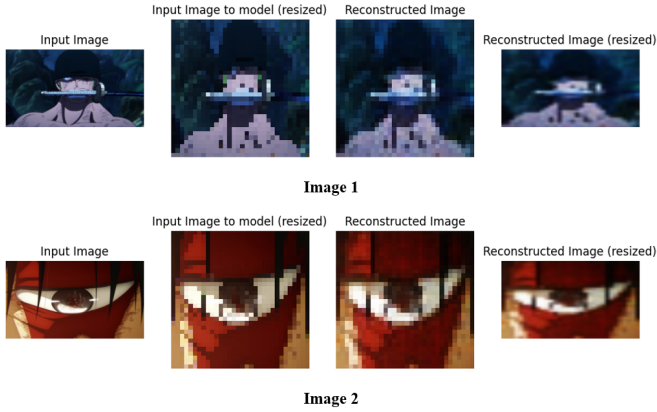


Fig. 13. Comparison of 2 custom test images between Original and Reconstructed images for ResNet50



Fig. 14. SRGAN performance outcomes on test data

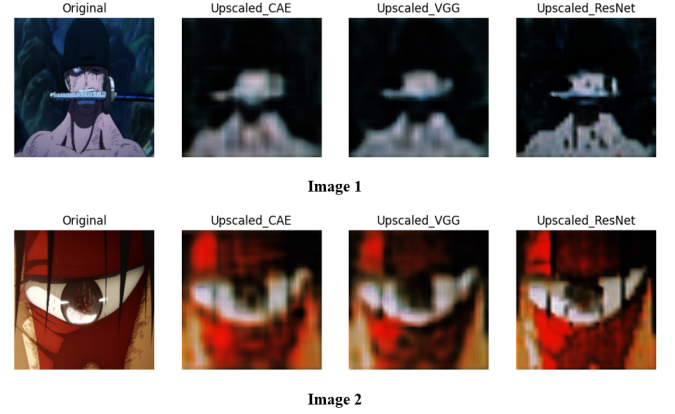


Fig. 15. Comparison of 3 custom test images between Original and Enhanced images for SRGAN

VI. CONCLUSION

Our paper explains about a methodology that helps in embedding features of an image in an image file and then reconstructing the image back using encoder decoder architecture. The approach commences with the encoder model which extracts semantic features of the image using three deep learning models CAE, VGG16 and Resnet-50. The steganography of the secret image in the carrier image file is done using LSB steganography technique. The decoder helps in reconstructing the original image back using the features extracted from the image. The best model for reconstruction was Resnet 50. There were 2 comparison done between the 3 models using the resized original image and reconstructed image and between original image and resized reconstructed image, these comparisons were for custom images. SRGAN was used further to enhance the quality of the reconstructed. Overall this study can be used as an approach for embedding images into image files via feature extraction and enhancement of the reconstruction via SRGAN.

REFERENCES

- [1] Zhenjie Bao and Ru Xue, "Research on the avalanche effect of image encryption based on the Cycle-GAN," Appl. Opt. 60, 5320-5334 (2021)

- [2] Liu, Z., Xue, R. Visual image encryption based on compressed sensing and Cycle-GAN. *Vis Comput* (2023)
- [3] Panwar, K.; Singh, A.; Kukreja, S.; Singh, K.K.; Shakhovska, N.; Boichuk, A. Encipher GAN: An End-to-End Color Image Encryption System Using a Deep Generative Model. *Systems* 2023
- [4] Kuyoro, A., Nzenwata, U.J., Awodele, O., Idowu, S. (2022). GAN-based encoding model for reversible image steganography. *Revue d'Intelligence Artificielle*, Vol. 36, No. 4, pp. 561-567.
- [5] Ramaneti, K., Kakani, P., Krishna, C., Rajkumar, S. (2021). Image Steganography Using GANs. In: Lee, R. (eds) *Computer and Information Science* 2021
- [6] Y. Ding, F. Tan, Z. Qin, M. Cao, K. -K. R. Choo and Z. Qin, "DeepKeyGen: A Deep Learning-Based Stream Cipher Generator for Medical Image Encryption and Decryption," in *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 9, pp. 4915-4929, Sept. 2022
- [7] Singh, O.D., Dhall, S., Malik, A. et al. A robust and secure immensely random GAN based image encryption mechanism. *Multimed Tools Appl* 82, 19693–19743 (2023).
- [8] Chahar, Vijay & Sharma, Sahil & Kumar, Chandan & Sahu, Aditya Kumar. "Latest Trends in Deep Learning Techniques for Image Steganography", *International Journal of Digital Crime and Forensics*, 2023
- [9] Q. Li et al., "A Novel Grayscale Image Steganography Scheme Based on Chaos Encryption and Generative Adversarial Networks," in *IEEE Access*, vol. 8, pp. 168166-168176, 2020
- [10] Monu Singh, Naman Baranwal, K.N. Singh, A.K. Singh, Huiyu Zhou, Deep learning-based biometric image feature extraction for securing medical images through data hiding and joint encryption-compression, *Journal of Information Security and Applications*, Volume 79, 2023
- [11] Qin, J.; Wang, J.; Tan, Y.; Huang, H.; Xiang, X.; He, Z. Coverless Image Steganography Based on Generative Adversarial Network. *Mathematics* 2020
- [12] V. Sharma, M. Shukla, S. Srivastava and R. Mandal, "Generative Network Based Image Encryption," 2020 4th International Conference on Intelligent Computing and Control Systems (ICICCS), Madurai, India, 2020
- [13] Yu, C., Hu, D., Zheng, S. et al. An improved steganography without embedding based on attention GAN. *Peer-to-Peer Netw. Appl.* 14, 1446–1457 (2021).
- [14] Yendluri Lohith JayaSurya , Yendluri Priya Ysaswini , Somepalli Saranya,"Image Steganography", *International Journal of Recent Technology and Engineering (IJRTE)*, Volume-9 Issue-3, September 2020
- [15] Yuan, C., Wang, H., He, P. et al. GAN-based image steganography for enhancing security via adversarial attack and pixel-wise deep fusion. *Multimed Tools Appl* 81, 6681–6701 (2022).
- [16] P. Mohan, P. B. Menon, P. K. Rahul and K. S. Sidharth, "Image Steganography-A New Approach using Block Truncation Coding and LSB Embedding," 2022 6th International Conference on Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 2022
- [17] M. Kataria, K. Jain and N. Subramanian, "Exploring Advanced Encryption and Steganography Techniques for Image Security," 2023 11th International Symposium on Digital Forensics and Security (ISDFS), Chattanooga, TN, USA, 2023
- [18] Alejandro Martín, Alfonso Hernández, Moutaz Alazab, Jason Jung, David Camacho, "Evolving Generative Adversarial Networks to improve image steganography", *Expert Systems with Applications*, Volume 222, 2023
- [19] Hafiz Abdul Rehman, Usama Ijaz Bajwa, Rana Hammad Raza, Sultan Alfarhood, Mejdl Safran, Fan Zhang, "Leveraging coverless image steganography to hide secret information by generating anime characters using GAN", *Expert Systems with Applications*, Volume 248, 2024
- [20] Ye Yao, Junyu Wang, Qi Chang, Yizhi Ren, Weizhi Meng, "High invisibility image steganography with wavelet transform and generative adversarial network", *Expert Systems with Applications*, 2024
- [21] Dewang Wang, Gaobo Yang, Jiyu Chen, Xiangling Ding, "GAN-based adaptive cost learning for enhanced image steganography security", *Expert Systems with Applications*, Volume 249, Part A, 2024